Introduction to Software Development Process

Zhiming Liu
zhiming.liu@swu.edu.cn
http://www.swu-rise.net.cn/zhiming.liu
Centre for Research and Innovation in Software Engineering
Southwest University
China

Development Process

CoCoME Case Study

System Design

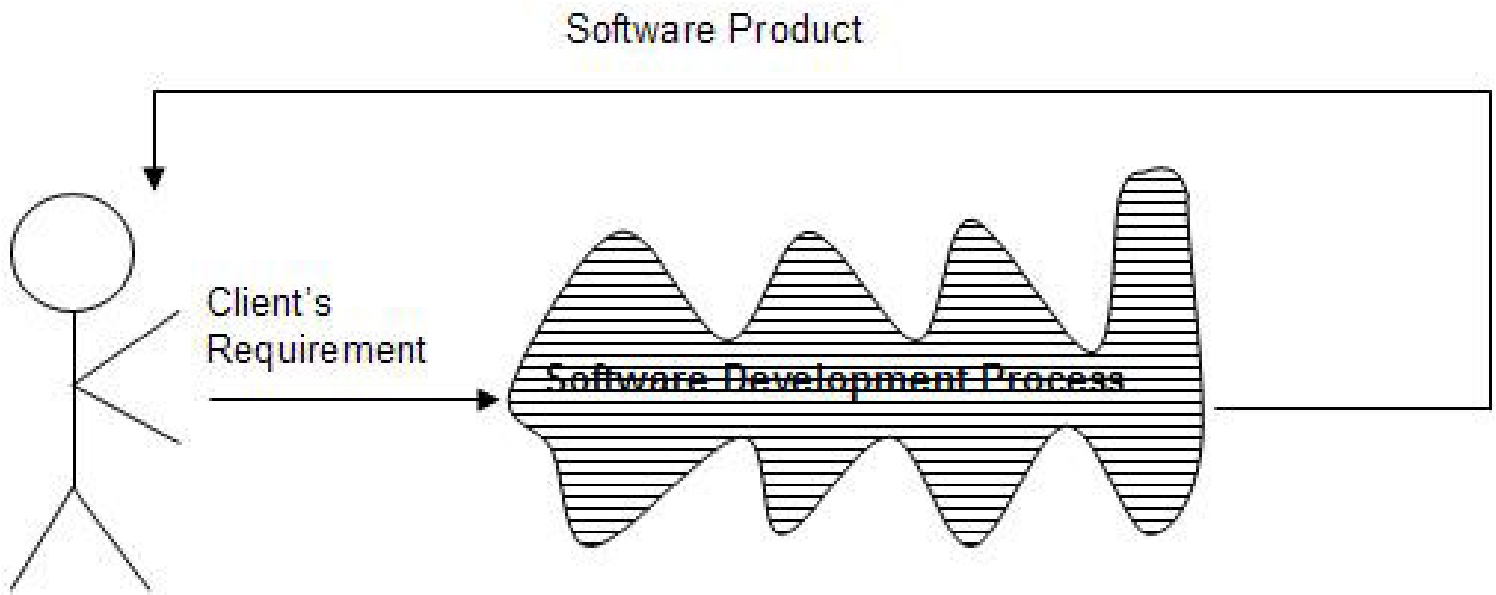
Implementation and Testing

Integration and System Testing

Waterfall Model

Software Development Process

- A **process** defines **who** is doing **what**, **when** and **how** to reach a certain goal
- An **engineering project** is about how to produce a product in a disciplined process
- A process to produce a software product is called a **SDP**
- **SE** is about producing software products in a disciplined process



Client's Requirements

- define the **goal** of the process
- prepared by the client and set out the **services** that the system is expected to provide
- include **functional requirements** and **non-functional constraints**

CoCoME: create a *trading system* to be used in an enterprise of supermarkets.

- **Goal**: increased checkout automation, to support faster, better and cheaper services and business processes
  - quick checkout for the customer
  - fast and accurate sales analysis
  - automatic inventory control
  - ...

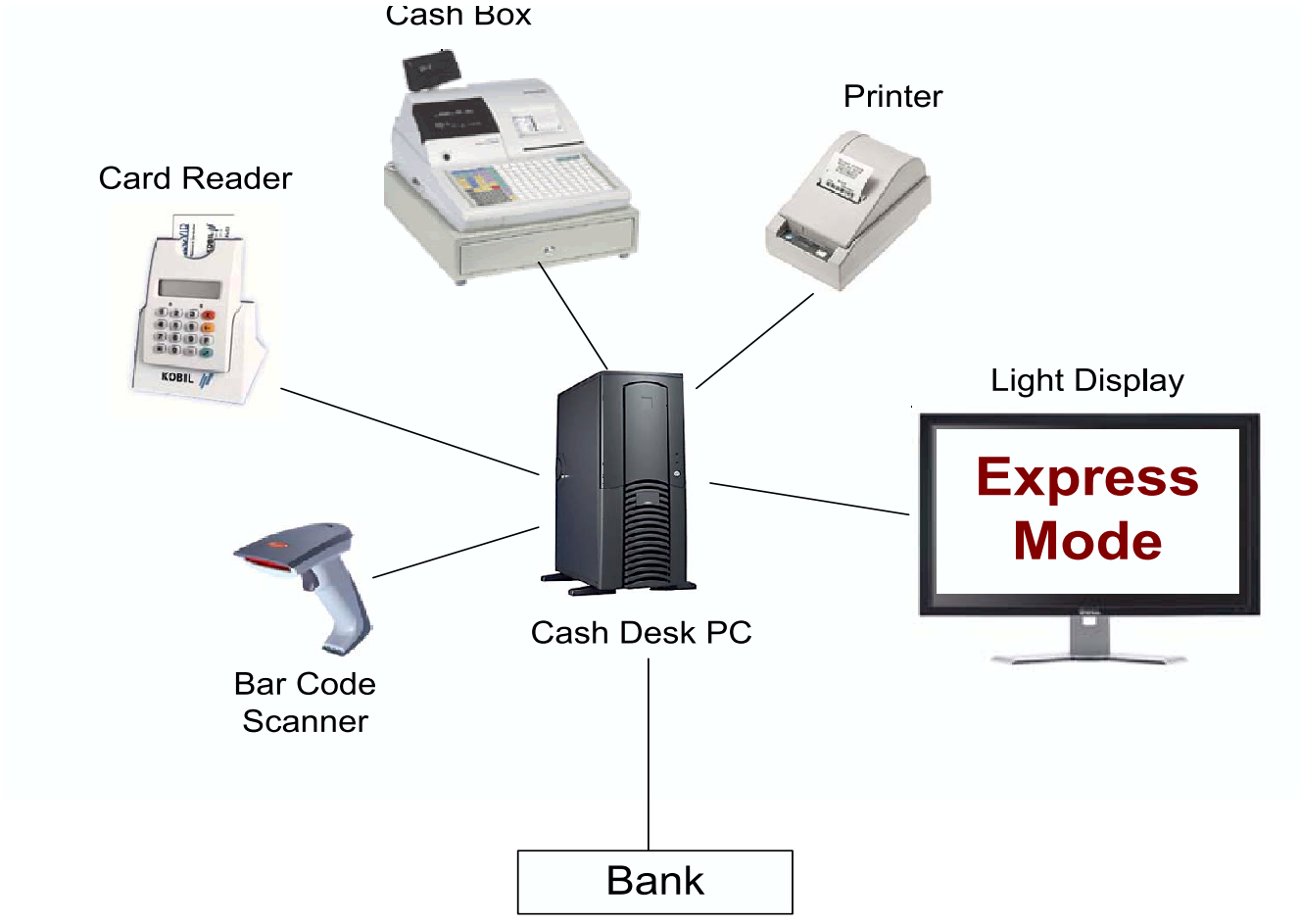
Client is a primary stakeholder of the project

Requirements of CoCoME

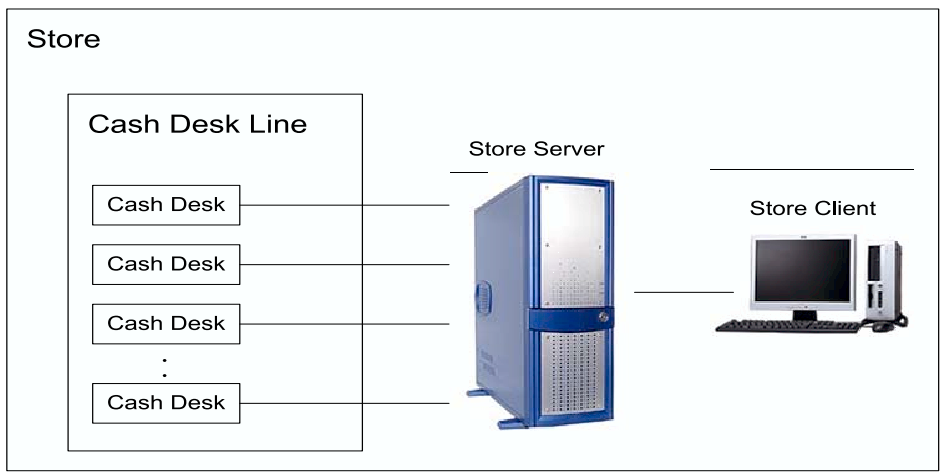
- about 20 pages long
- description of **hierarchy of components**
  - **cash desk**: cash box, bar code scanner, card reader, printer, lights, cash desk PC
  - **store**: a line of cash desks, store server
  - **enterprise**: a net works of stores, enterprise server
- **interactions** with the **bank** and **supplier**
- **functional requirements**: 8 use cases: **process sale** ...
- **extra functionality**: QoS

Difficult to be complete in one go

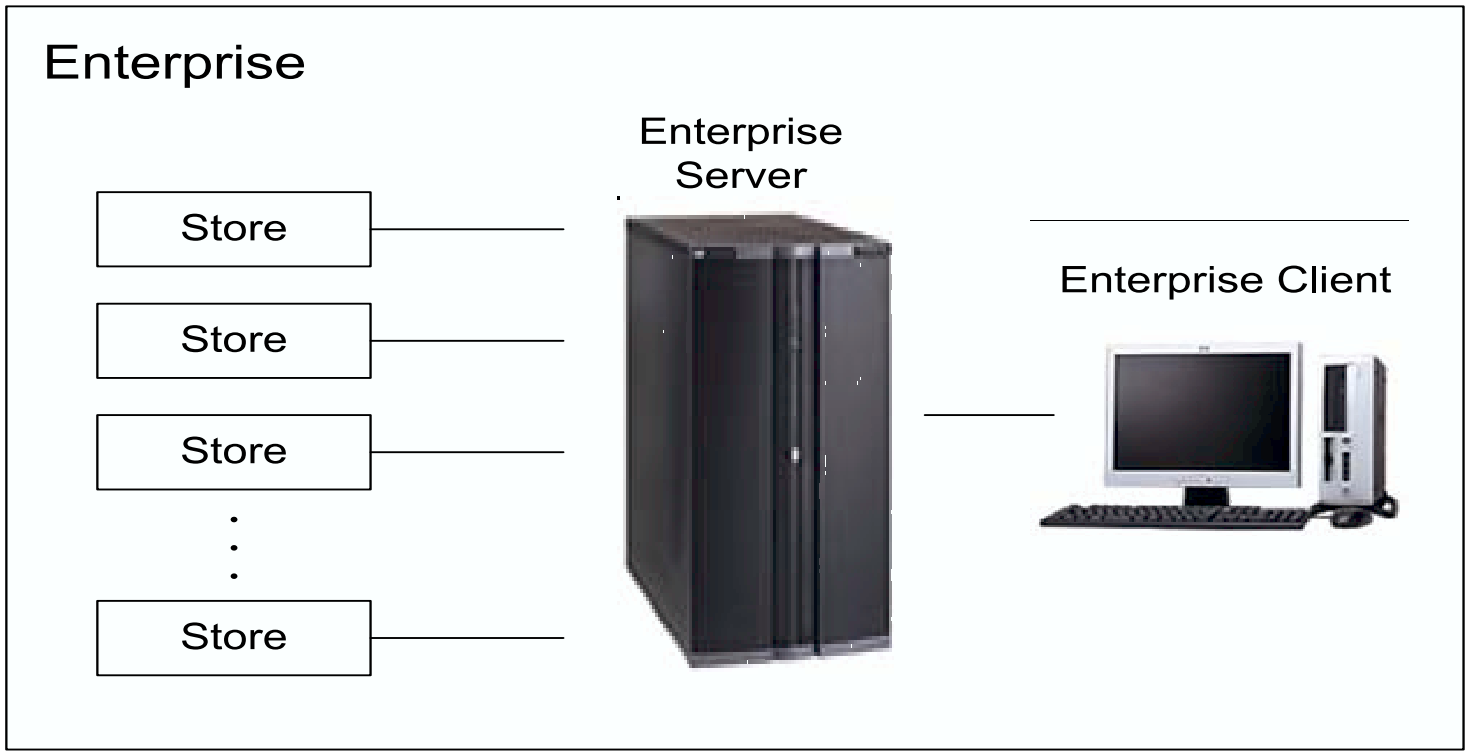
Cash Desk


Cash Box
Printer
Card Reader
Light Display
Express Mode
Cash Desk PC
Bar Code Scanner
Bank

Store


Store
Cash Desk Line
Cash Desk
Cash Desk
Cash Desk
Cash Desk
Store Server
Store Client

Enterprise


Enterprise
Enterprise Server
Store
Store
Store
Store
Enterprise Client

Problems with Client's Requirements

- described in terms of concepts and objects from the application domain
- not be directly understandable to software engineers
- unstructured, incomplete and contain inconsistency
- may not even be feasible
- cannot be used for precise RA and VV

SDP must start with understanding and capturing the client requirements, called **requirement capture and analysis**

Requirement Capture and Analysis

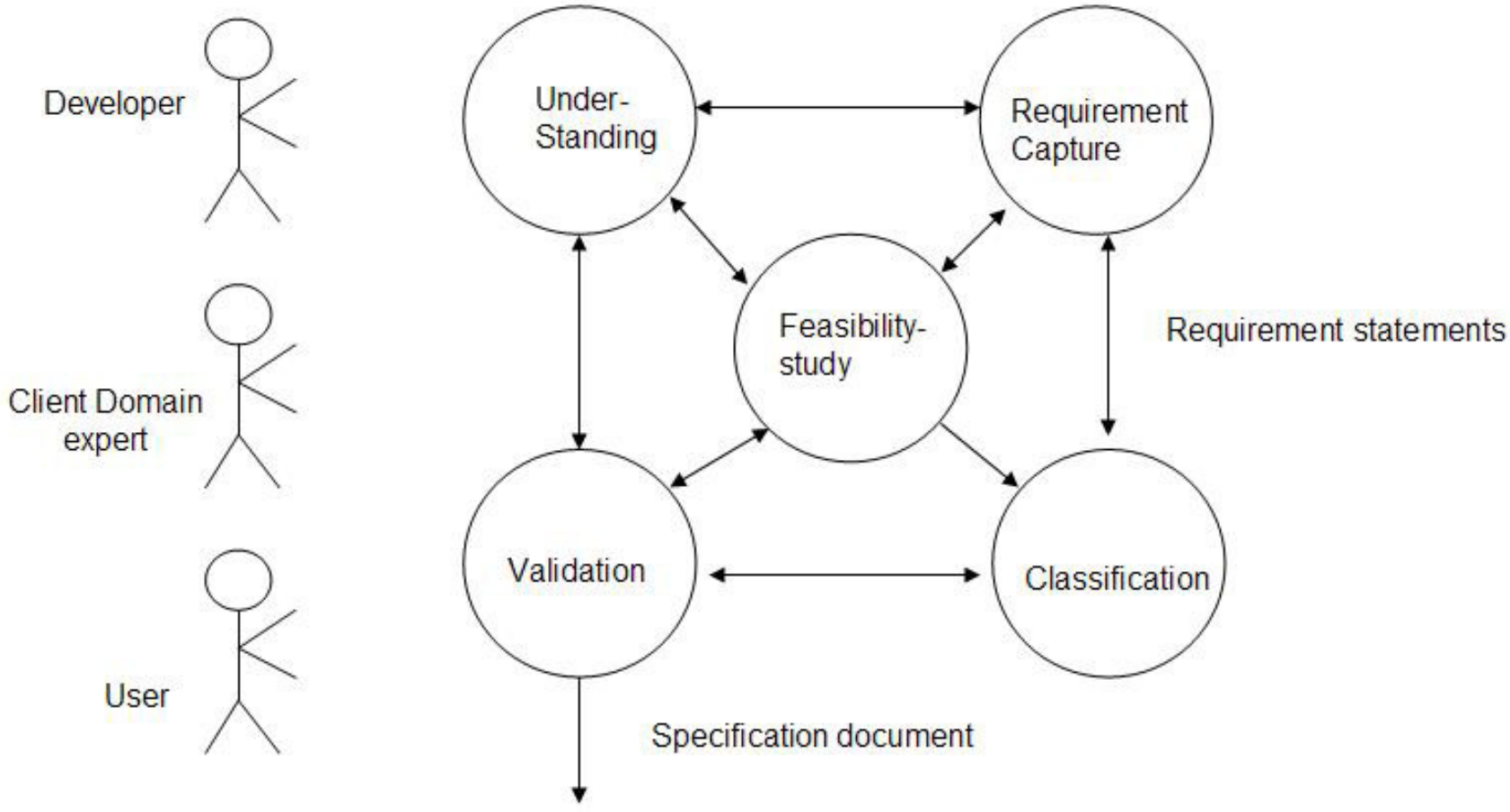
Aim to produce a document called **requirements specification**

- is a fairly full model of what is required of the system
- state **what to be done** rather than **how it is done**
- used as
  1. the agreed contract between the client and the system development organisation
  2. the basis used by the development team to design the system correctly
  3. the basis by which the quality of the end-product is measured

RA Activities

- **Domain understanding**: explore concept and develop understanding of the application domain
- **Requirements capture** interact with the client, and the application domain experts, and end users to discover and capture their requirements
- **Classification**: organise and structure the requirements captured into coherent clusters
- **Validation**: check consistency and completeness, and to resolve conflicting requirements
- **Feasibility study** estimate if the requirements may be satisfied using the soft/hardware technologies, to decide if the system cost-effect
- Undiscovered errors are likely to remain undiscovered
- The later an error is discovered, the more expensive is it to fix

RA Activities and Artifacts



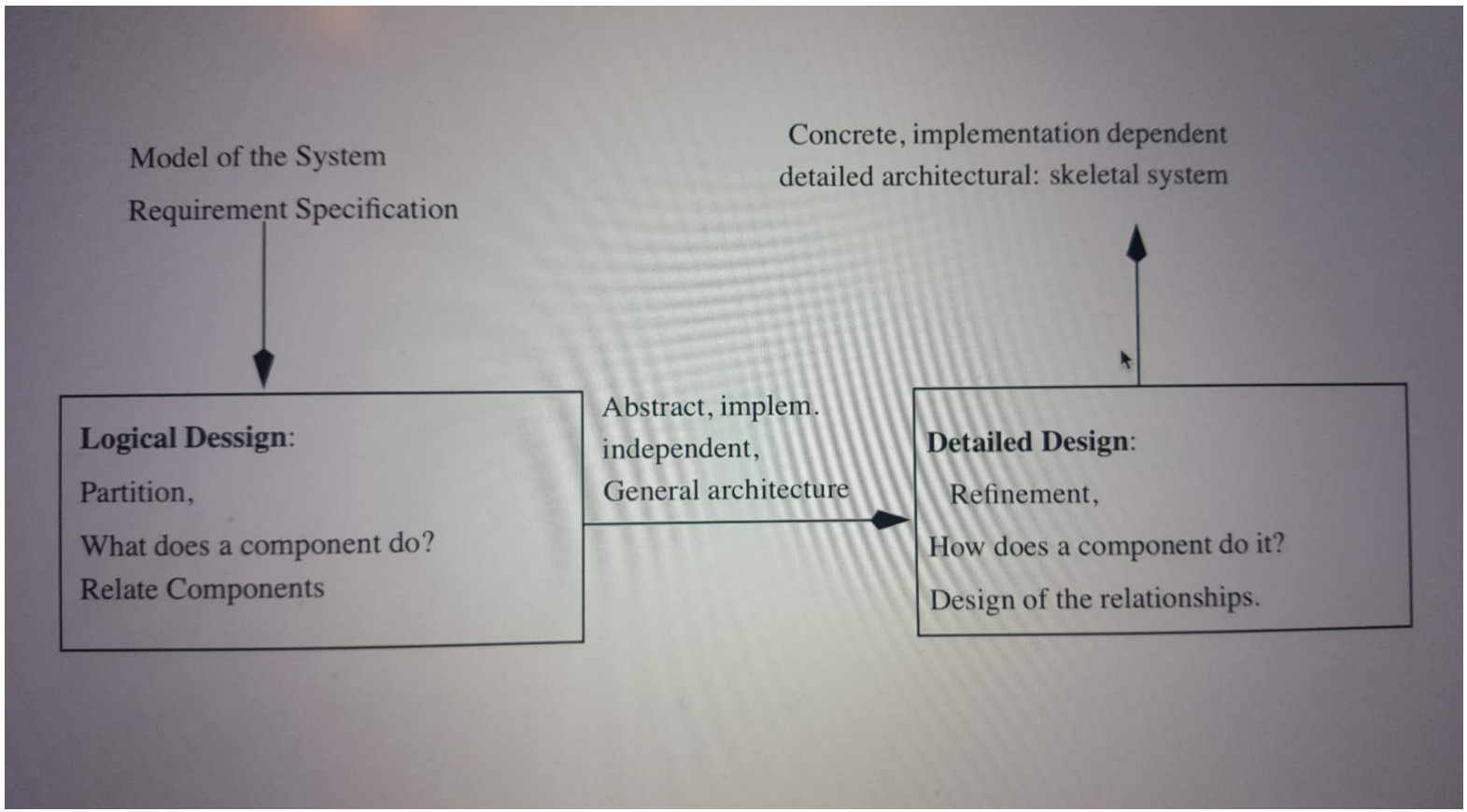
When is RA Completed

- No hard rules!
- Ask the following questions before taking the SDP into the next phase
  - Has the system required been fully understood by the client, end-users, and the developers?
  - Has a fairly complete model of what is required built?
    - What functions (or services) are available?
    - What the input & output are;
    - What data are necessary?
  - But no implementation decisions should be described in this model
- The requirements specification of the system must be adjusted and improved once found necessary during a later stage of the development
- Requirements of software for modern networked based systems always are ever changing as systems are continuously evolving

System Design

After the requirements are analysed, the requirement specification undergoes two consecutive design processes

1. architectural (or logical) design, and
2. detailed (or physical) design.



Architecture Design

The requirements specification is decomposed into *components* to produce an architectural document (or model) which describes

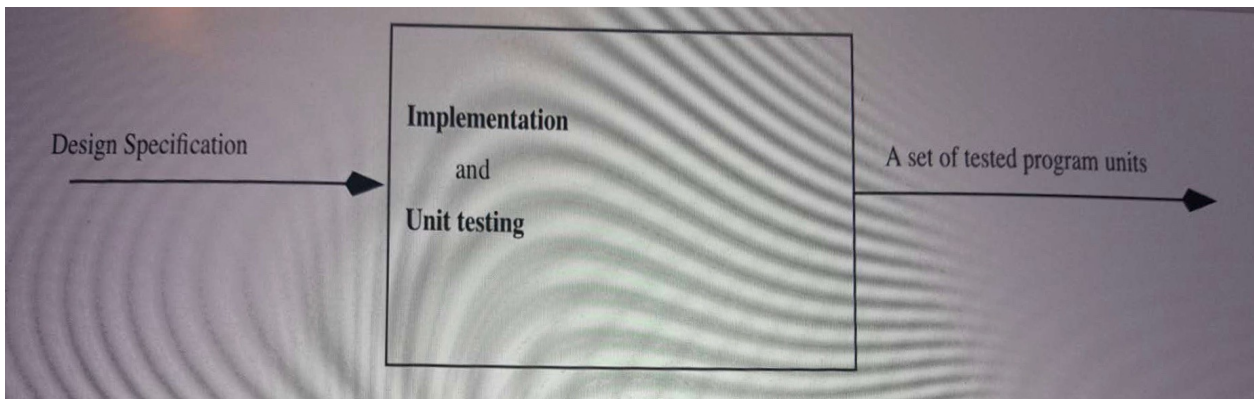
- what each component must do ? components specifications
- what the relations between components are - *components interfaces* specifications

Detailed Design

- Refines the components specifications to produce a *detailed design* model which describes
  - the design of the functions of each component: *how* the component does the function - algorithms,
  - the design of the interfaces for the components: *how* components interact with each other - protocols
- This model can be seen as a skeletal system which is concrete and ready for implementation

Implementation and Testing

- Each of the components from the design is realized as a program unit
  - Note that component can be composite or hyerachical
- Each unit then must be either verified or tested against its specification obtained in the design stage



- Test plan must be made according to the decomposition in the logical design specification.
- Test cases must be design to cover functions of all components

Integration and System Testing

- The individual program units representing the components of the system are combined
- The system is tested as a whole to ensure that the software requirements have been met
- The system is tested by the client (acceptance testing)
- This phase ends when the product is accepted by the client

Integration and System Testing

- The individual program units representing the components of the system are combined
- The system is tested as a whole to ensure that the software requirements have been met
- The system is tested by the client (acceptance testing)
- This phase ends when the product is accepted by the client

NB

1. Test plan must be made according to the requirement specification
2. Test cases should be designed to cover all the crucial services required

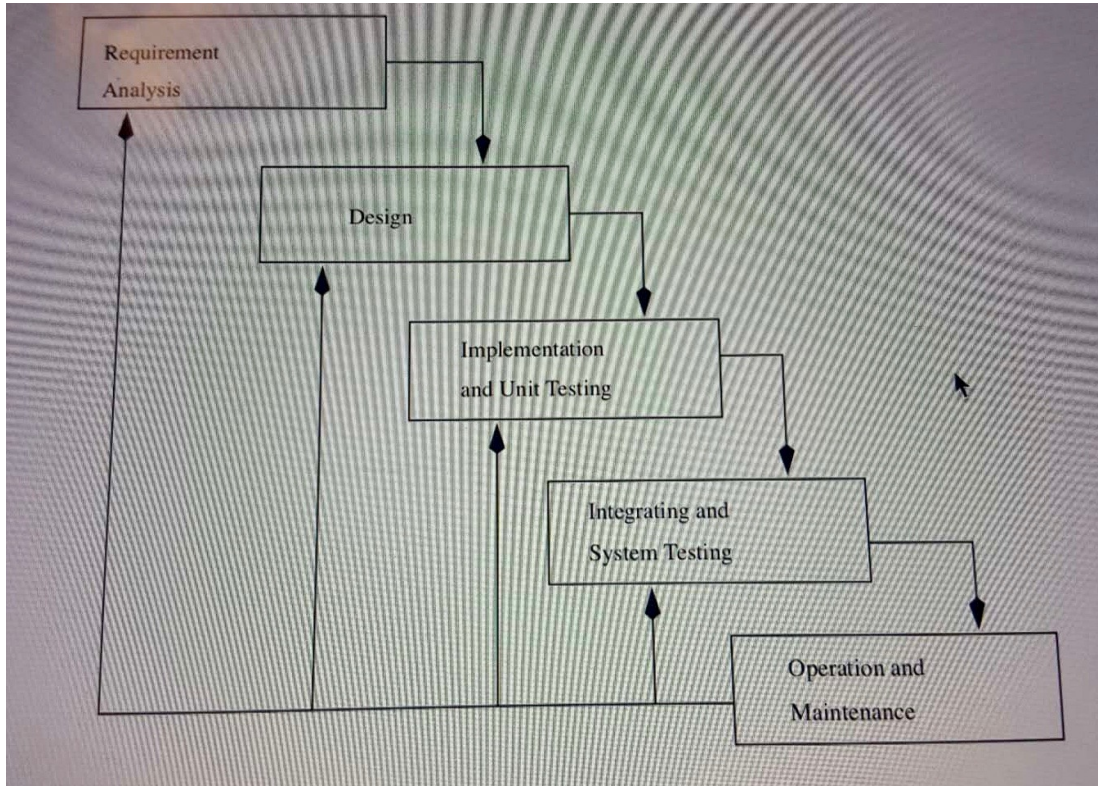
Operation and Maintenance

- Starts with the system being installed for practical use after accepted by the client
- Lasts till the beginning of the system?s retirement phase

Operation and Maintenance

- Starts with the system being installed for practical use after accepted by the client
- Lasts till the beginning of the system?s retirement phase
- Maintenance includes all changes to the product after it is accepted by the client
  - Corrective Maintenance involves correcting errors which were not discovered earlier while leaving the specification unchanged.
  - Perfective maintenance involves changes that the client thinks will improve the effectiveness of the product - **enhancement**.
  - Adaptive maintenance are about changes made in response to changes in the environment in which the product operates, or to new - **update**
  - Extension/evolution

Waterfall Model



Problems of the Waterfall Model

Often

- A project is difficult to be partitioned into the distinct stages of requirement analysis, design and so on.
- It is also difficult or impossible to establish a detailed and complete requirement specification – all modern network based systems.
- E.G. Develop an AI system which attempt to emulate some human capabilities.
  - As we do not understand how humans carry out tasks, setting out a detailed specification for software to imitate humans is impossible
- Evolutionary: Iterative and Incremental development, but
  - Waterfall model is used in each iteration